Institut Supérieur du Numérique (SupNum)

Année académique : 2024 / 2025

RAPPORT DE PROJET INTÉGRATEUR DU 5E SEMESTRE

Madrasati

Système de Gestion Scolaire

Plateforme Web Full-Stack pour la Gestion Administrative
et Pédagogique des Établissements d'Enseignement Primaire

Présenté par :

Zeini Cheikh Sidi Ely *23025*

Ahmed Mohamed Lemin *23028*

Mohamed Vall Kbeiduche *23041*

El Mokhtar N'Gourane *23035*

Idoumou M. Abdellahi *22020*

Encadré par :

M. Sidi Mahmoud KABER

Dédicaces

À nos familles, pour leur soutien sans faille tout au long de ce parcours.

À nos enseignants, pour les connaissances transmises avec passion.

À tous ceux qui ont contribué, de près ou de loin, à ce projet.

Remerciements

Nous tenons à exprimer notre sincère gratitude envers toutes les personnes qui ont contribué à la réalisation de ce projet intégrateur.

Nos remerciements s'adressent en premier lieu à notre encadrant, **M. Sidi Mahmoud KABER**, pour sa disponibilité constante, ses orientations précieuses et le soin qu'il a apporté au suivi de ce travail. Son expertise et sa rigueur ont été déterminants dans l'avancement et la qualité du projet.

Nous remercions également l'ensemble du corps enseignant de l'**Institut Supérieur du Numérique (SupNum)** pour la qualité de la formation académique et professionnelle, ainsi que pour les méthodes et outils transmis.

Nos remerciements vont aussi à nos camarades de promotion pour l'entraide qui a marqué ce semestre. Enfin, nous exprimons toute notre reconnaissance à nos familles pour leur soutien moral inestimable.

Résumé

Ce rapport présente **Madrasati**, une plateforme web de gestion scolaire développée dans le cadre du Projet Intégrateur du 5e Semestre à l'Institut Supérieur du Numérique (SupNum). Le projet répond au besoin de centraliser et d'automatiser la gestion administrative et pédagogique des établissements d'enseignement primaire.

La solution repose sur une architecture trois tiers : un frontend avec React 19 et TypeScript, un backend RESTful avec Django et Django REST Framework, et une base de données PostgreSQL. L'application intègre l'authentification JWT, un contrôle d'accès par rôle (administrateur, enseignant, secrétaire), un workflow d'approbation des résultats et un tableau de bord statistique. L'interface est bilingue arabe/français.

Mots-clés : Gestion scolaire, Application web, React, Django, REST API, JWT, Architecture 3-tiers.

Contents

Remerciements **ii**

Résumé **iii**

Liste des figures **vi**

Liste des tableaux **vii**

Introduction générale **1**

1 Analyse et Spécification des Besoins **3**

1.1 Présentation du projet 3

1.1.1 Contexte et motivation 3

1.1.2 Périmètre fonctionnel 3

1.2 Identification des acteurs 3

1.3 Besoins fonctionnels 4

1.3.1 Authentification 4

1.3.2 Gestion des classes 4

1.3.3 Gestion des élèves 4

1.3.4 Gestion des matières 4

1.3.5 Gestion des résultats 4

1.3.6 Statistiques et tableau de bord 4

1.4 Besoins non fonctionnels 5

1.5 Cas d'utilisation 5

2 Conception et Architecture **7**

2.1 Architecture 3-tiers 7

2.2 Modélisation des données 8

2.3 Schéma de base de données 9

2.4 Règles métier 10

3 Technologies et Environnement de Développement **11**

3.1 Stack technique 11

3.1.1 Frontend 11

3.1.2 Backend 11

3.1.3 Outils 12
3.2 Justification des choix 12

4 Réalisation et Implémentation **13**
4.1 Backend 13
4.1.1 Modèles et persistance 13
4.1.2 Endpoints de l'API 13
4.2 Frontend 14
4.2.1 Connexion et tableau de bord 14
4.2.2 Gestion des élèves 15
4.2.3 Classes et matières 16
4.2.4 Résultats et approbation 17
4.2.5 Statistiques et import 18

5 Tests et Validation **19**
5.1 Stratégie 19
5.2 Tests d'intégration API 19
5.3 Tests fonctionnels 20
5.4 Intégration continue 20

Conclusion et perspectives **21**

Références bibliographiques **22**

A Guide d'installation **23**
A.1 Prérequis 23
A.2 Backend 23
A.3 Frontend 23

B Variables d'environnement **24**

List of Figures

1.1 Diagramme de cas d'utilisation global 5

2.1 Architecture 3-tiers du système Madrasati 7
2.2 Diagramme de classes UML 8
2.3 Schéma ERD de la base de données 9
2.4 Workflow d'approbation des résultats 10
2.5 Hiérarchie des composants React 10

4.1 Page de connexion – Interface bilingue 14
4.2 Tableau de bord – Indicateurs clés 14
4.3 Liste des élèves avec filtrage 15
4.4 Formulaire de fiche élève 15
4.5 Gestion des classes 16
4.6 Gestion des matières 16
4.7 Saisie des résultats – Vue enseignant 17
4.8 Approbation des résultats – Vue administrateur 17
4.9 Tableau de bord statistique 18
4.10 Import en masse – Fichier Excel 18

List of Tables

1.1 Acteurs du système et leurs responsabilités 3
1.2 Besoins non fonctionnels du système 5
1.3 Cas d'utilisation principal : Gérer les résultats 6

2.1 Patterns architecturaux par couche 7
2.2 Formules de calcul des moyennes 10

3.1 Technologies du frontend 11
3.2 Technologies du backend 11
3.3 Outils de développement 12

4.1 Endpoints de l'API REST 13

5.1 Résultats des tests d'intégration API 19
5.2 Résultats des tests fonctionnels 20

A.1 Prérequis d'installation 23

B.1 Variables d'environnement du backend 24

Introduction Générale

Contexte

La transformation numérique touche aujourd'hui tous les secteurs, et l'éducation n'en fait pas exception. Dans les établissements scolaires primaires, la gestion reste souvent manuelle : fichiers dispersés, registres papier, absence de workflow de validation. Cette situation engendre des erreurs, des pertes de données et un manque de visibilité sur les performances scolaires.

Problématique

- La dispersion des données entre différents fichiers sans cohérence globale ;
- L'absence de processus formalisé pour la validation des résultats ;
- Le manque d'outils de pilotage statistique par classe et par matière ;
- L'absence de support bilingue adapté au contexte mauritanien.

Objectifs

Madrasati vise à développer une plateforme web unifiée qui centralise la gestion scolaire, sécurise les accès par rôle, automatise le calcul des moyennes et offre un tableau de bord en temps réel.

Structure du rapport

Chapitre 1 – Analyse et Spécification des Besoins
Identification des acteurs, exigences fonctionnelles et non fonctionnelles, cas d'utilisation.

Chapitre 2 – Conception et Architecture
Architecture 3-tiers, diagrammes UML, schéma de base de données, règles métier.

Chapitre 3 – Technologies et Environnement de Développement
Présentation et justification des technologies, frameworks et outils utilisés.

Chapitre 4 – Réalisation et Implémentation
Description des modules développés, de l'API REST et des interfaces utilisateur.

Chapitre 5 – Tests et Validation
Stratégie de test, résultats obtenus et bilan de la validation du système.

1. Analyse et Spécification des Besoins

1.1 Présentation du projet

1.1.1 Contexte et motivation

Madrasati (« mon école » en arabe) est une application web développée à SupNum dans le cadre du Projet Intégrateur du 5e Semestre. Elle couvre les niveaux 1 à 6 de l'enseignement primaire et gère l'ensemble du cycle de vie d'un élève, depuis son inscription jusqu'à la publication de ses résultats.

1.1.2 Périmètre fonctionnel

- Gestion des utilisateurs avec contrôle d'accès par rôle ;
- Gestion des classes, des élèves et des matières ;
- Saisie, soumission et approbation des résultats selon un workflow ;
- Calcul automatique des moyennes pondérées ;
- Tableau de bord statistique interactif ;
- Import/export Excel.

1.2 Identification des acteurs

Acteur	**Rôle et responsabilités**
Administrateur	Accès complet. Gère les comptes, valide ou rejette les résultats, consulte les statistiques.
Enseignant	Saisit les notes (contrôles et examens), consulte les stats de ses classes, soumet pour validation.
Secrétaire	Gère les inscriptions et données des élèves. Importe les listes en masse depuis Excel.

Table 1.1: Acteurs du système et leurs responsabilités

1.3 Besoins fonctionnels

1.3.1 Authentification

Connexion sécurisée via JWT : Access Token (1 h) + Refresh Token (7 jours). Accès conditionné par le rôle.

1.3.2 Gestion des classes

Création, modification et suppression de classes (niveaux 1–6), filtrage, comptage des élèves.

1.3.3 Gestion des élèves

Fiches complètes : nom, naissance, genre, téléphone parent, adresse. Rattachement à une classe. Import Excel en masse.

1.3.4 Gestion des matières

Définition par niveau avec nombre de points total servant de coefficient.

1.3.5 Gestion des résultats

- Saisie par type (contrôle / examen) et semestre (1, 2 ou 3) ;
- Validation que la note ne dépasse pas le total de la matière ;
- Workflow : *en attente* → *approuvé* / *rejeté* (motif obligatoire) ;
- Export Excel + impression des bulletins.

1.3.6 Statistiques et tableau de bord

Moyenne générale, taux de réussite, performances par classe/matière, graphiques filtrables par semestre.

1.4 Besoins non fonctionnels

Critère	**Exigence**
Sécurité	JWT, RBAC, validation côté serveur, 100 req/min max.
Performance	Réponse API $<$ 500 ms pour les opérations courantes.
Maintenabilité	Architecture modulaire, séparation stricte des couches.
Internationalisation	Arabe et français natif, bascule à la volée.
Compatibilité	Chrome, Firefox, Edge – versions récentes.
Portabilité	Linux + Python 3.10+ + Node.js 18+.

Table 1.2: Besoins non fonctionnels du système

1.5 Cas d'utilisation

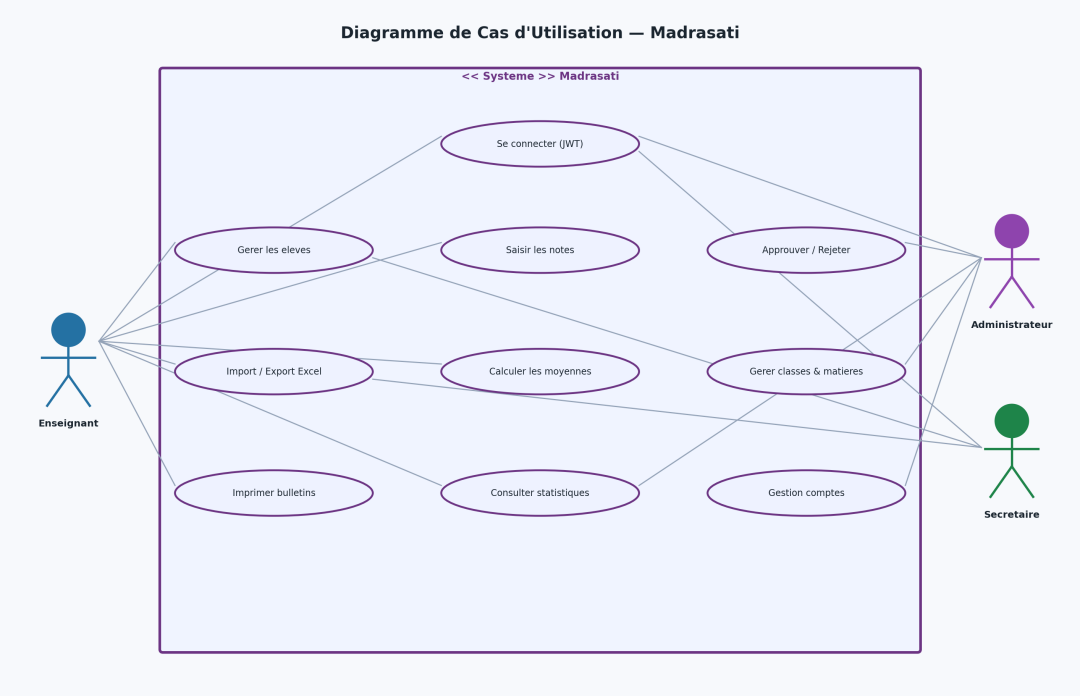

Figure 1.1: Diagramme de cas d'utilisation global

Cas d'utilisation : Gérer les résultats scolaires	
Acteurs	Enseignant (initiateur), Administrateur (validateur)
Préconditions	Utilisateur authentifié. Élèves, classes et matières existants.
Scénario	1. Sélection classe + semestre. 2. Saisie des notes. 3. Soumission → statut *en attente*. 4. Administrateur consulte et approuve / rejette avec motif.
Postconditions	Résultats approuvés intégrés aux statistiques.
Exception	Note > total de points : saisie refusée.

Table 1.3: Cas d'utilisation principal : Gérer les résultats

2. Conception et Architecture

2.1 Architecture 3-tiers

L'architecture retenue sépare le système en trois couches indépendantes. Ce choix favorise la maintenabilité, la testabilité et l'évolutivité.

Présentation (React 19/TypeScript) : interface et interactions. **Métier** (Django+DRF) : logique, validation, droits. **Données** (PostgreSQL/SQLite via ORM) : persistance.

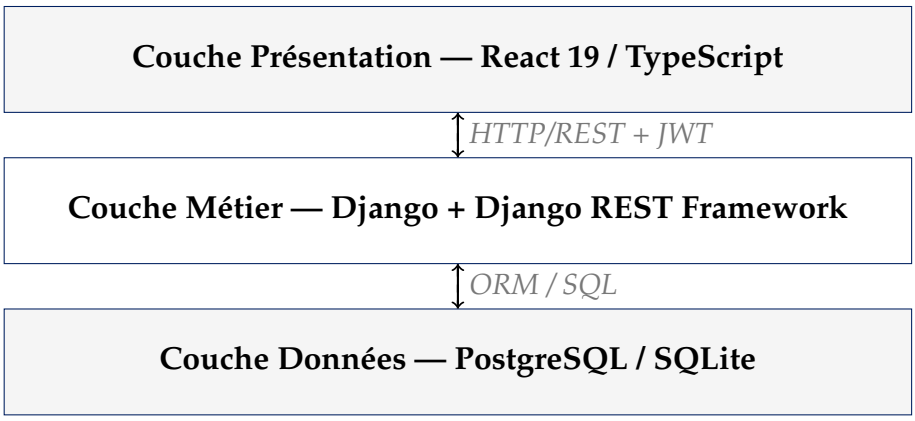

Figure 2.1: Architecture 3-tiers du système Madrasati

Couche	Pattern utilisé
Frontend	Context API, structure feature-based, Custom Hooks
Backend	Model-View-Serializer (MVS), ViewSets CRUD
Authentification	JWT avec rotation automatique des Refresh Tokens

Table 2.1: Patterns architecturaux par couche

2.2 Modélisation des données

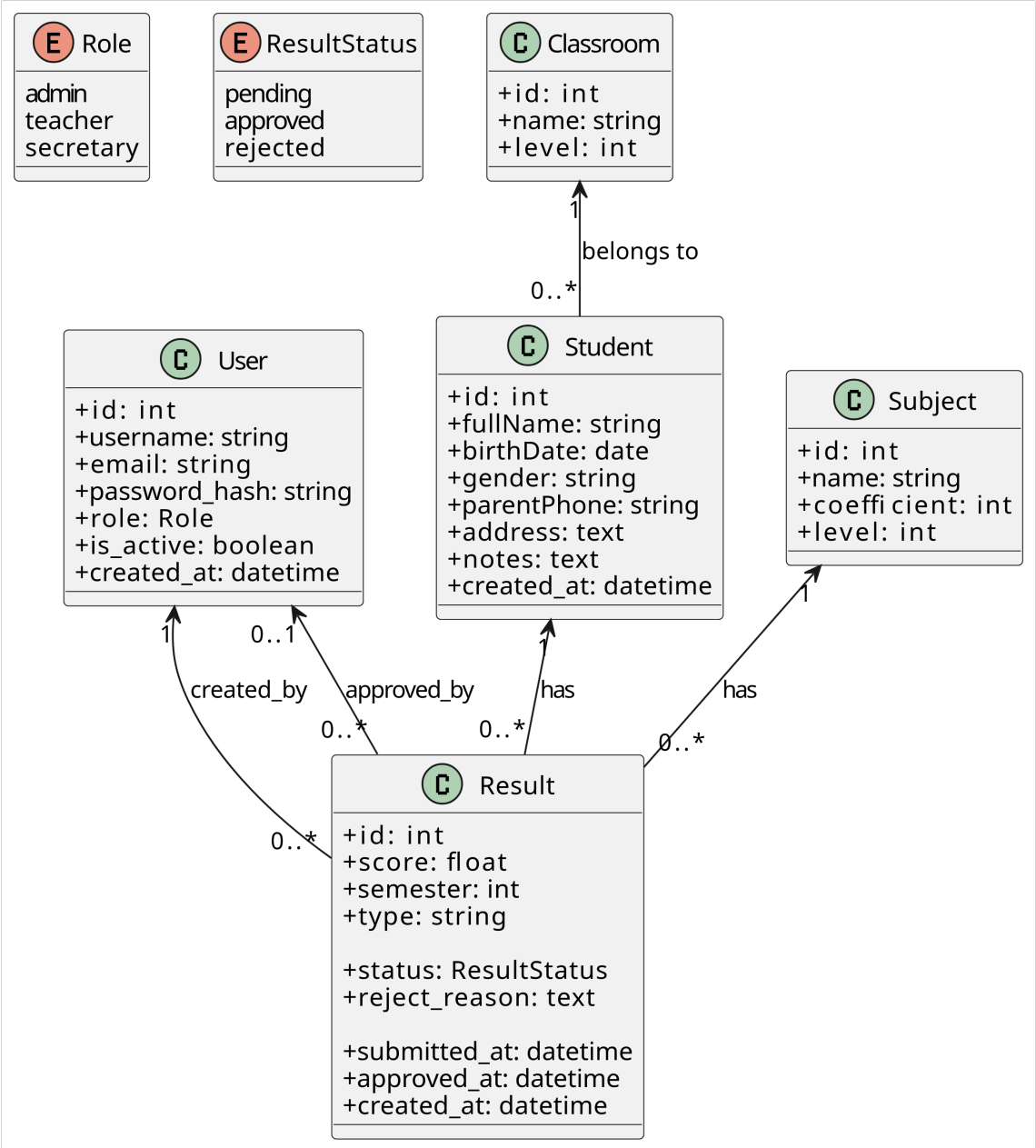

Figure 2.2: Diagramme de classes UML

Classroom : groupe-classe (nom + niveau 1–6). **Student** : élève rattaché à une classe (nom, naissance, genre, tél., adresse). **Subject** : matière d'un niveau avec un total de points servant de coefficient. **Result** : note d'un élève par matière/semestre/type, avec statut workflow et motif de rejet éventuel. Contrainte d'unicité sur (élève, matière, semestre, type).

Relations : Classroom 1–N Student (cascade), Student 1–N Result, Subject 1–N Result.

2.3 Schéma de base de données

Figure 2.3: Schéma ERD de la base de données

2.4 Règles métier

Situation	**Formule**
Contrôle + Examen	$\text{Moy.} = \dfrac{\text{contrôle} + \text{examen}}{2}$
Examen seul	$\text{Moy.} = \text{note examen}$
Contrôle seul	$\text{Moy.} = \text{note contrôle}$
Moyenne générale	$\bar{M} = \dfrac{\sum_i \text{moy}_i \times \text{pts}_i}{\sum_i \text{pts}_i}$
Seuil de réussite	$\bar{M} \geq 10$

Table 2.2: Formules de calcul des moyennes

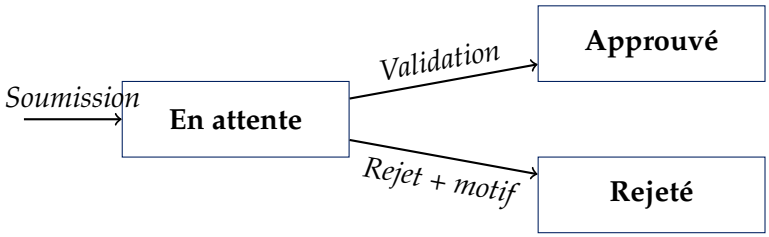

Figure 2.4: Workflow d'approbation des résultats

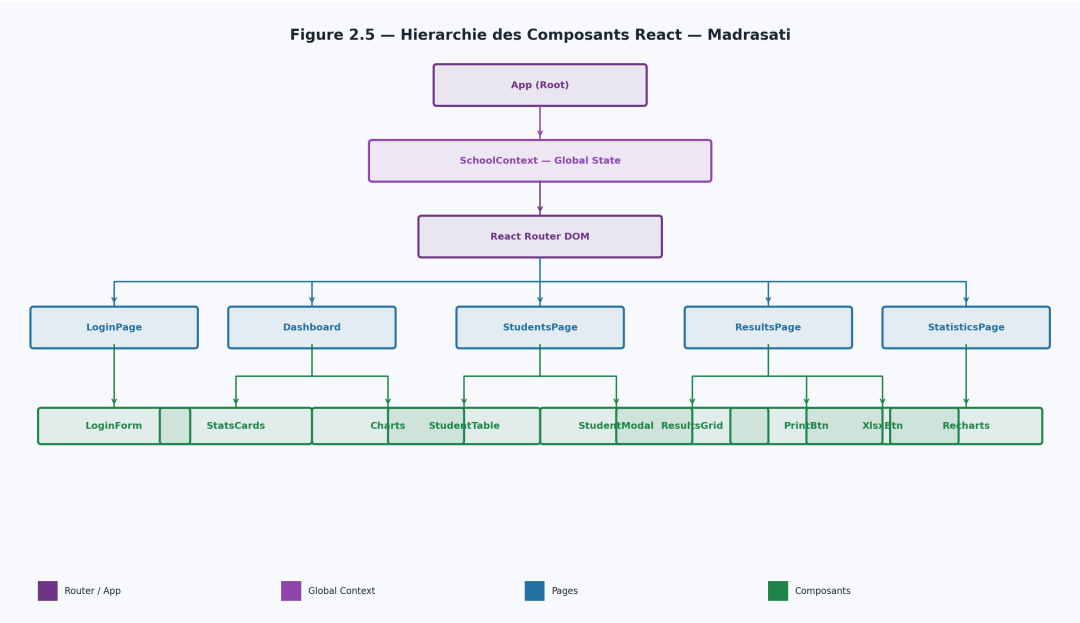

Figure 2.5: Hiérarchie des composants React

3. Technologies et Environnement de Développement

3.1 Stack technique

3.1.1 Frontend

Technologie	Version	Rôle
React	19.2.1	Construction d'interfaces utilisateur
TypeScript	5.8.2	Typage statique
Vite	6.2.0	Bundler + serveur de développement
React Router DOM	7.10.1	Routage côté client
Recharts	3.5.1	Graphiques statistiques interactifs
xlsx	–	Import/export Excel
i18next	25.x	Internationalisation arabe/français
Tailwind CSS	3.4.19	CSS utilitaire

Table 3.1: Technologies du frontend

3.1.2 Backend

Technologie	Version	Rôle
Django	$\geq$6.0	Framework web Python + ORM
Django REST Framework	$\geq$3.14	API REST (ViewSets, Serializers)
simplejwt	$\geq$5.3	Authentification JWT
django-cors-headers	$\geq$4.3	Gestion CORS
psycopg2-binary	$\geq$2.9	Adaptateur PostgreSQL
python-dotenv	$\geq$1.0	Variables d'environnement

Table 3.2: Technologies du backend

3.1.3 Outils

Outil	**Usage**
Git / GitHub	Versioning et collaboration
VS Code	Éditeur de code
Postman	Tests des endpoints API
GitHub Actions	CI/CD automatisé

Table 3.3: Outils de développement

3.2 Justification des choix

React 19 + TypeScript : composants réutilisables, DOM virtuel performant, typage réduisant les erreurs. **DRF** : ViewSets génériques, Serializers pour validation+sérialisation, permissions intégrées. **JWT** : authentification stateless, scalabilité horizontale, rotation automatique des tokens.

4. Réalisation et Implémentation

4.1 Backend

4.1.1 Modèles et persistance

Les modèles Django définissent la structure BDD avec validations intégrées (plages de valeurs, unicité, règles référentielles).

4.1.2 Endpoints de l'API

Ressource	**URL**	**Opérations**
Authentification	`/api/token/`	Login, refresh
Élèves	`/api/students/`	CRUD, import
Classes	`/api/classes/`	CRUD, filtre niveau
Matières	`/api/subjects/`	CRUD, filtre niveau
Résultats	`/api/results/`	CRUD, approbation/rejet
Statistiques	`/api/statistics/`	Lecture, filtre semestre

Table 4.1: Endpoints de l'API REST

4.2 Frontend

4.2.1 Connexion et tableau de bord

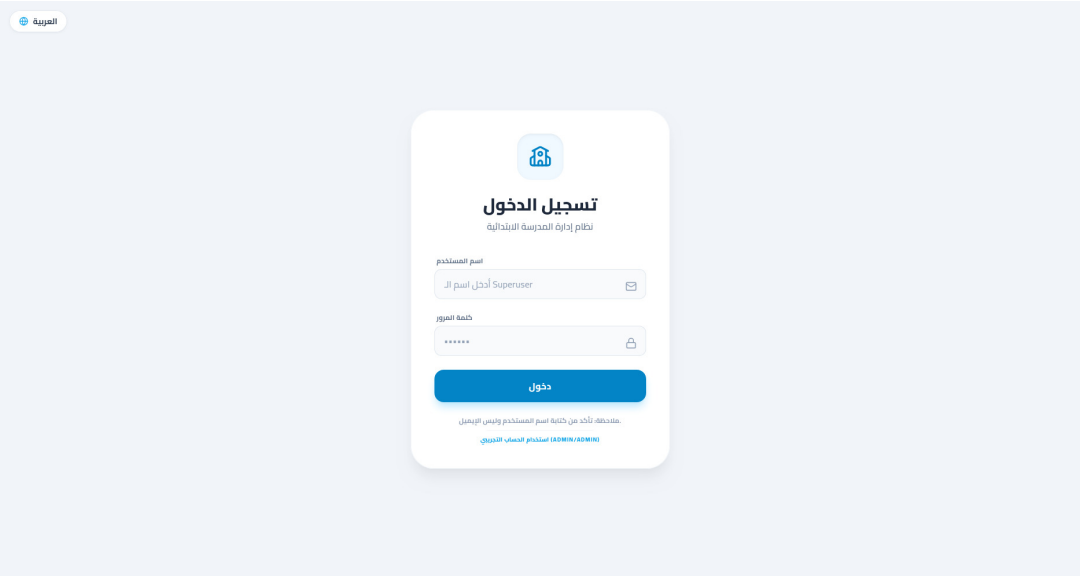

Figure 4.1: Page de connexion – Interface bilingue

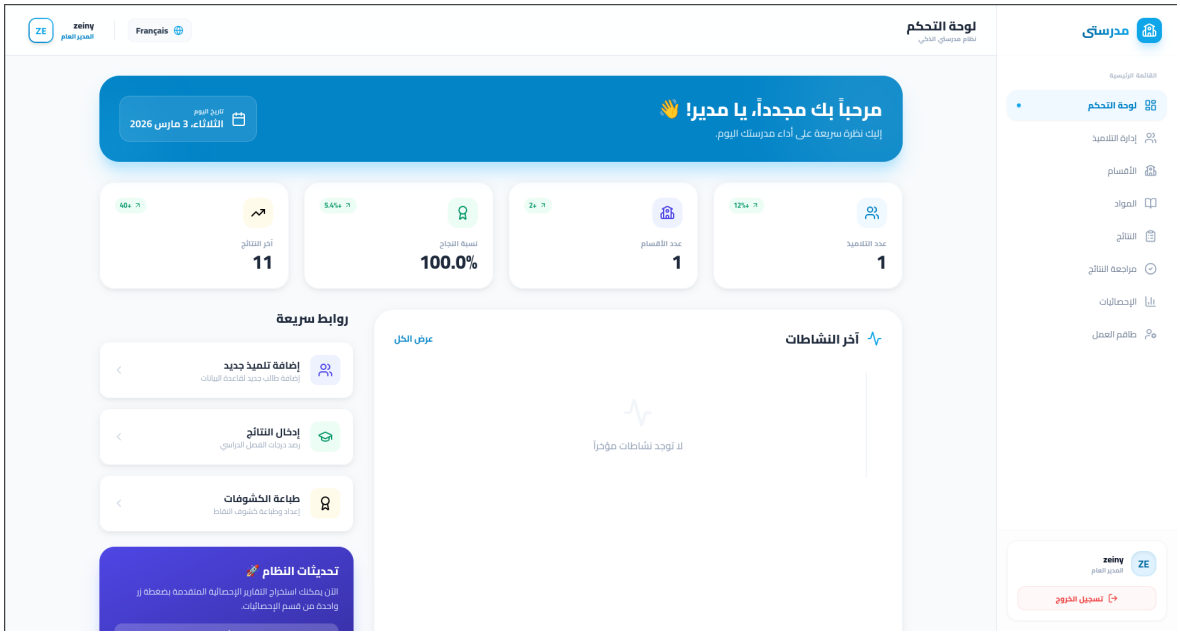

Figure 4.2: Tableau de bord – Indicateurs clés

4.2.2 Gestion des élèves

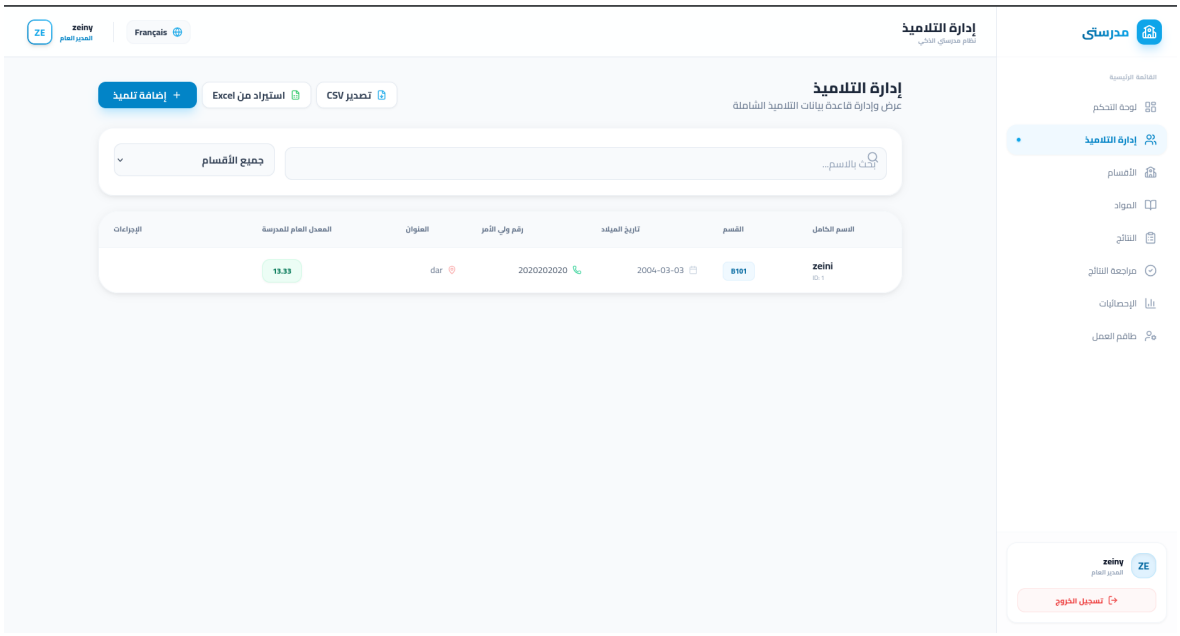

Figure 4.3: Liste des élèves avec filtrage

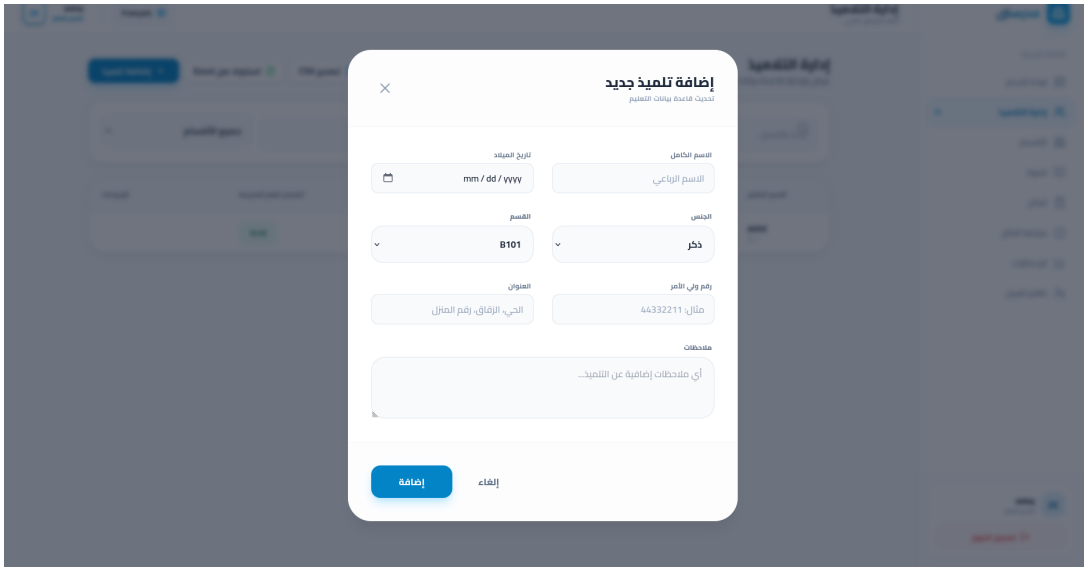

Figure 4.4: Formulaire de fiche élève

4.2.3 Classes et matières

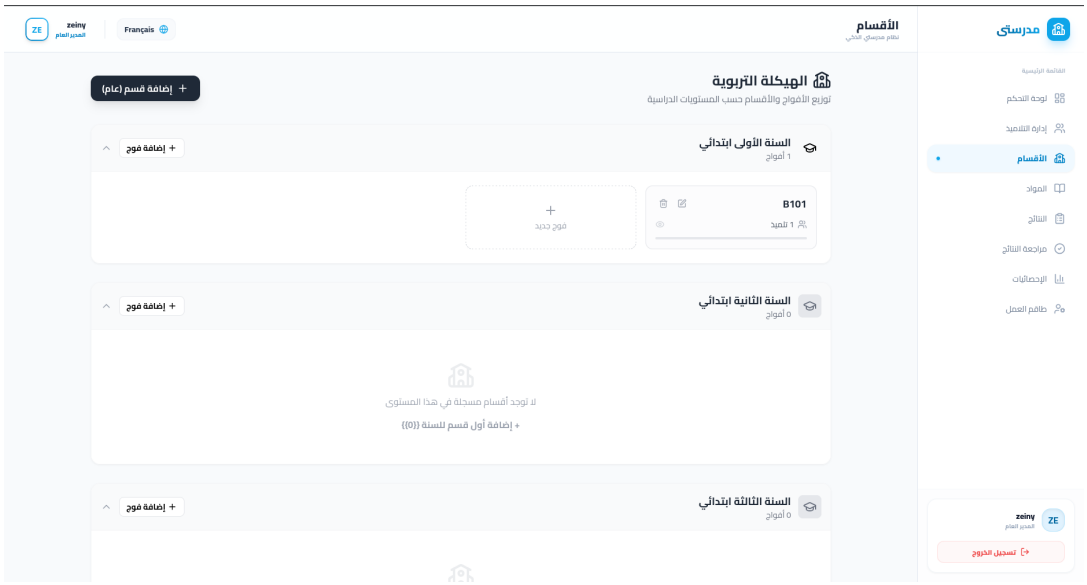

Figure 4.5: Gestion des classes

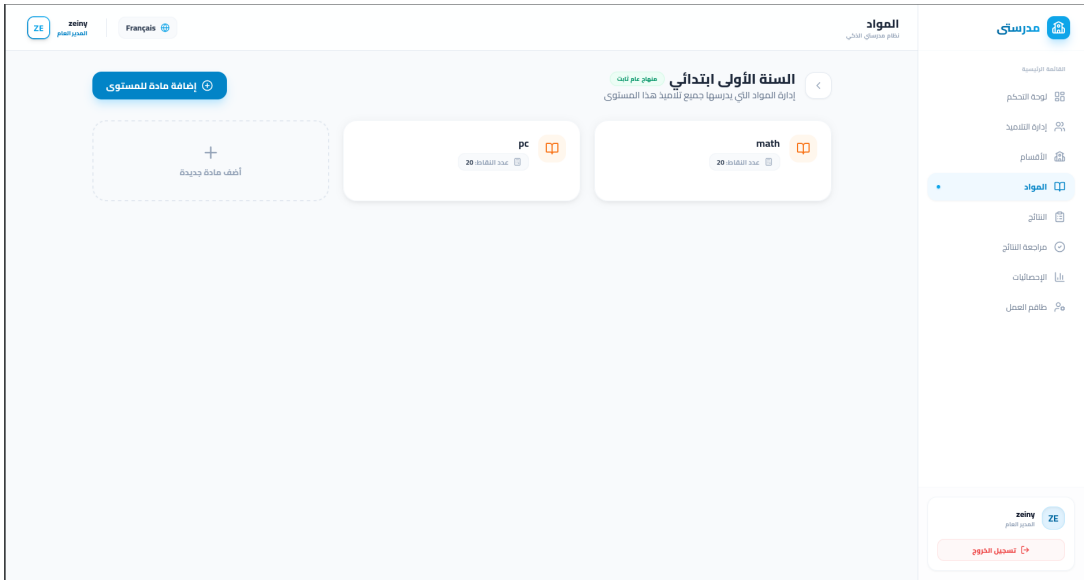

Figure 4.6: Gestion des matières

4.2.4 Résultats et approbation

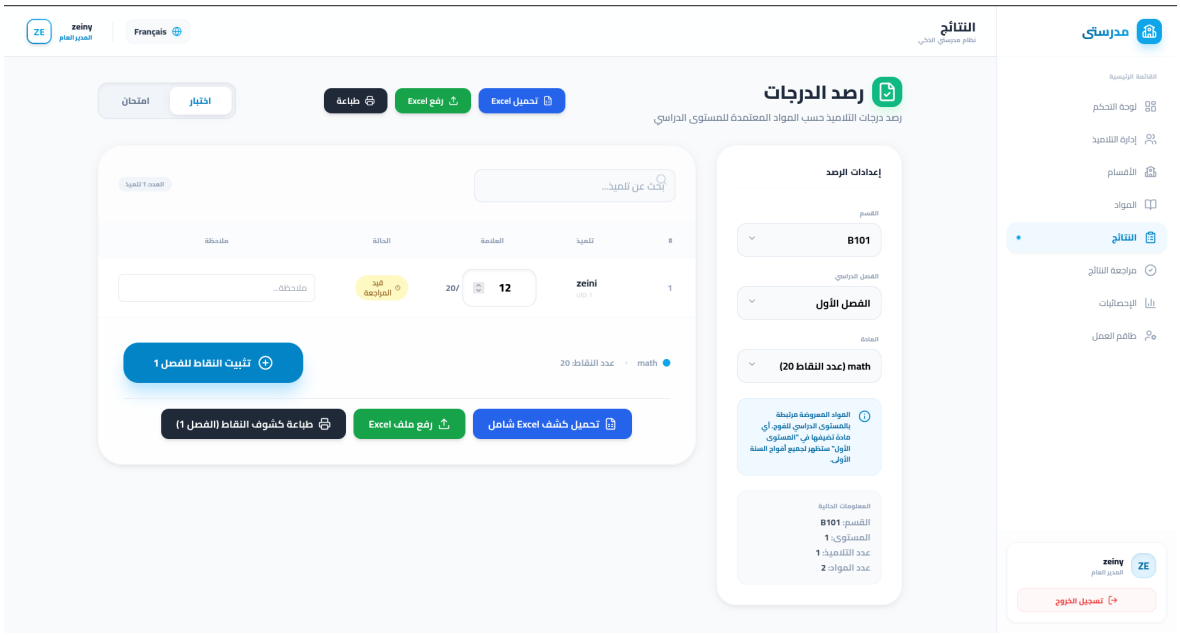

Figure 4.7: Saisie des résultats – Vue enseignant

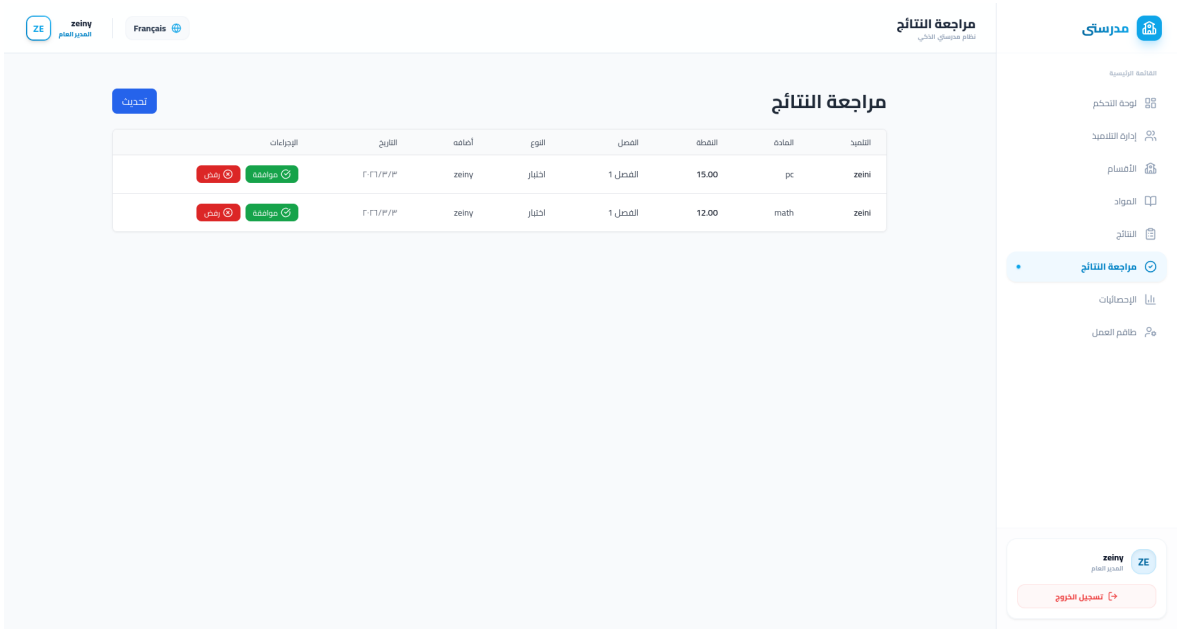

Figure 4.8: Approbation des résultats – Vue administrateur

4.2.5 Statistiques et import

Figure 4.9: Tableau de bord statistique

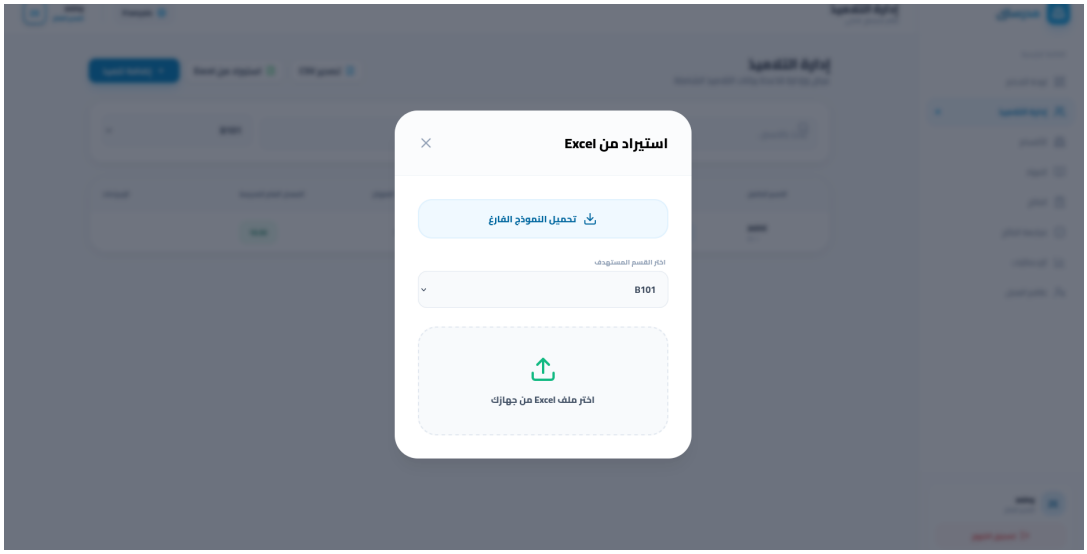

Figure 4.10: Import en masse – Fichier Excel

5. Tests et Validation

5.1 Stratégie

Quatre niveaux : tests **unitaires** (modèles, calculs), **intégration** (flux API), **fonctionnels** (interfaces par profil), **sécurité** (RBAC).

5.2 Tests d'intégration API

Endpoint	**Méthode**	**Code attendu**	**Résultat**
`/api/token/`	POST	200	✓ Conforme
`/api/students/`	GET	200	✓ Conforme
`/api/students/`	POST	201	✓ Conforme
`/api/results/`	POST	201	✓ Conforme
`/api/results/{id}/approve/`	POST	200	✓ Conforme
`/api/statistics/`	GET	200	✓ Conforme
`/api/students/` (sans auth)	GET	401	✓ Conforme

Table 5.1: Résultats des tests d'intégration API

5.3 Tests fonctionnels

Scénario	**Description**	**Résultat**
Connexion par rôle	Chaque profil accède à son espace uniquement	✓
Ajout élève	Création confirmée, visible dans la liste	✓
Import Excel	50 élèves importés sans erreur	✓
Saisie notes	Soumission réussie	✓
Approbation	Résultats intégrés aux stats	✓
Rejet	Motif enregistré et affiché	✓
Statistiques	Graphiques cohérents	✓
Langue	Bascule sans rechargement	✓

Table 5.2: Résultats des tests fonctionnels

5.4 Intégration continue

GitHub Actions exécute automatiquement les tests backend à chaque push, garantissant la non-régression et signalant toute anomalie.

Conclusion et Perspectives

Bilan

Le projet **Madrasati** a abouti à une plateforme de gestion scolaire complète et fonctionnelle. Tous les objectifs ont été atteints : centralisation des données, automatisation des calculs, workflow d'approbation, tableau de bord et interface bilingue. Ce projet a permis de mettre en pratique les compétences acquises à SupNum : architecture logicielle, conception UML, développement full-stack, sécurisation d'API et tests.

Perspectives

- Containerisation Docker pour simplifier le déploiement ;
- Cookies `HttpOnly` pour renforcer la sécurité JWT ;
- Notifications temps réel via WebSocket ;
- Génération de bulletins PDF officiels ;
- Module de gestion des absences ;
- Application desktop multi-plateforme (Windows, macOS, Linux) basée sur **Electron**, permettant une utilisation hors ligne et une intégration native avec le système d'exploitation ;
- Application mobile React Native ;
- Suivi individuel de progression par élève.

Références Bibliographiques

[1] Django Software Foundation. *Django Documentation*. https://docs.djangoproject.com/

[2] Tom Christie. *Django REST Framework*. https://www.django-rest-framework.org/

[3] Meta. *React Documentation v19*. https://react.dev/

[4] Evan You. *Vite Documentation*. https://vitejs.dev/

[5] Auth0. *Introduction to JSON Web Tokens*. https://jwt.io/introduction

[6] Microsoft. *TypeScript Handbook*. https://www.typescriptlang.org/docs/

[7] Adam Wathan. *Tailwind CSS*. https://tailwindcss.com/docs/

[8] PostgreSQL Group. *PostgreSQL Documentation*. https://www.postgresql.org/docs/

[9] OpenJS Foundation. *Electron Documentation*. https://www.electronjs.org/docs/

[10] M. Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

[11] I. Sommerville. *Software Engineering*, 10e ed. Pearson, 2015.

A. Guide d'installation

A.1 Prérequis

Composant	Version minimale
Python	3.10+
Node.js	18+
npm	9+
PostgreSQL	14+ (production)

Table A.1: Prérequis d'installation

A.2 Backend

1. Créer un environnement virtuel Python et l'activer ;
2. `pip install -r requirements.txt` ;
3. Copier et configurer `.env` depuis `env_template.txt` ;
4. `python manage.py migrate` ;
5. `python manage.py createsuperuser` ;
6. `python manage.py runserver` → API sur `http://127.0.0.1:8000`.

A.3 Frontend

1. `npm install` ;
2. Définir `VITE_API_BASE_URL` dans `.env` ;
3. `npm run dev` → Interface sur `http://localhost:5173`.

B. Variables d'environnement

Variable	Défaut	Description
`SECRET_KEY`	–	Clé secrète Django
`DEBUG`	`True`	Développement uniquement
`DB_ENGINE`	`sqlite3`	Moteur BDD
`DB_NAME`	`school_db`	Nom de la BDD
`CORS_ALLOWED_ORIGINS`	`localhost:3000`	Origines CORS
`JWT_ACCESS_TOKEN_LIFETIME_HOURS`	`1`	Durée Access Token
`JWT_REFRESH_TOKEN_LIFETIME_DAYS`	`7`	Durée Refresh Token

Table B.1: Variables d'environnement du backend